# Quasi-Newton Methods

Last modified: 2026-11-06 15:40:58 (PST)

Source and license

This section is adapted from the quasi-Newton lecture of Zhou [1], used under the MIT License. The license text is:


> Copyright (c) 2025 ucla-biostat-257
>
> Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
>
> The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
>
> THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.


From the source, this section takes the order of ideas: the cost of Newton's method, the secant condition, and the BFGS and limited-memory BFGS updates. The source states the updates as solutions of nearest-matrix problems without proof; here the BFGS update is written out. The Davidon-Fletcher-Powell and rank-1 updates, the nearest-matrix derivation, the history, and the Julia code are not part of this section.

**Definition 0.1** (Secant condition)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, and let $\underset{\sim}{x}, \underset{\sim}{x}^+ \in \mathbb{R}^p$. Write $\underset{\sim}{s} \stackrel{\text{def}}{=} \underset{\sim}{x}^+ - \underset{\sim}{x}$ for the step and $\underset{\sim}{y} \stackrel{\text{def}}{=} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^+) - \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})$ for the change in the gradient. A symmetric matrix $\mathbf{B}$ satisfies the **secant condition** [2, Ch. 6] for $(\underset{\sim}{s}, \underset{\sim}{y})$ when

$$\mathbf{B}\underset{\sim}{s} = \underset{\sim}{y}.$$

**Example 0.1** (The Hessian of a quadratic function satisfies the secant condition)**.** Let $f(\underset{\sim}{x}) = \frac{1}{2}\underset{\sim}{x}^\top \mathbf{Q}\underset{\sim}{x} - \underset{\sim}{b} \cdot \underset{\sim}{x}$ with $\mathbf{Q}$ symmetric. Then $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) = \mathbf{Q}\underset{\sim}{x} - \underset{\sim}{b}$, so for any $\underset{\sim}{x}$ and $\underset{\sim}{x}^+$

$$\begin{aligned} \underset{\sim}{y} &= (\mathbf{Q}\underset{\sim}{x}^+ - \underset{\sim}{b}) - (\mathbf{Q}\underset{\sim}{x} - \underset{\sim}{b}) && \text{(gradient at each point)} \\ &= \mathbf{Q}(\underset{\sim}{x}^+ - \underset{\sim}{x}) && \text{(collect terms)} \\ &= \mathbf{Q}\underset{\sim}{s}. && \text{(definition of } \underset{\sim}{s}) \end{aligned}$$

So the Hessian $\mathbf{Q}$ of the quadratic function satisfies the secant condition for every step. For a general $f$, $\underset{\sim}{y} \approx \mathbf{H}_f \underset{\sim}{s}$ holds for a short step, with $\mathbf{H}_f$ the Hessian of $f$ near the step, which is why a matrix that satisfies the secant condition is a reasonable stand-in for the Hessian.

**Definition 0.2** (Quasi-Newton method)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, let $\underset{\sim}{x}^{(0)}$ be a starting point, and let $\mathbf{H}^{(0)}$ be a positive definite matrix. A **quasi-Newton method** [2, Ch. 6] produces, for $t = 0, 1, 2, \ldots$,

$$\underset{\sim}{x}^{(t+1)} \stackrel{\text{def}}{=} \underset{\sim}{x}^{(t)} - \alpha^{(t)} \mathbf{H}^{(t)} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)}),$$

where $\alpha^{(t)} > 0$ is a step length chosen by a line search, and $\mathbf{H}^{(t+1)}$ is a symmetric positive definite matrix whose inverse satisfies the secant condition, that is, $\mathbf{H}^{(t+1)}\underset{\sim}{y}^{(t)} = \underset{\sim}{s}^{(t)}$, with $\underset{\sim}{s}^{(t)} = \underset{\sim}{x}^{(t+1)} - \underset{\sim}{x}^{(t)}$ and $\underset{\sim}{y}^{(t)} = \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t+1)}) - \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)})$.

**Example 0.2** (One quasi-Newton step on a quadratic function)**.** Take $f(\underset{\sim}{x}) = \frac{1}{2}\underset{\sim}{x}^\top \mathbf{Q}\underset{\sim}{x} - \underset{\sim}{b} \cdot \underset{\sim}{x}$ with $\mathbf{Q} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$ and $\underset{\sim}{b} = (1, 2)^\top$ (Example 0.1), $\underset{\sim}{x}^{(0)} = \underset{\sim}{0}$ and $\mathbf{H}^{(0)} = \mathbf{I}_2$. For a quadratic function, the step length that minimizes $f$ along a direction $\underset{\sim}{d}$ is $\alpha = -\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) \cdot \underset{\sim}{d} / \underset{\sim}{d}^\top \mathbf{Q}\underset{\sim}{d}$.

```
q_entries <- c(4, 1, 1, 3)
Q <- matrix(q_entries, nrow = 2)
b <- c(1, 2)
x0 <- c(0, 0)
H0 <- diag(2)
q_x0 <- drop(Q %*% x0)
grad0 <- q_x0 - b
d <- -drop(H0 %*% grad0)
q_d <- drop(Q %*% d)
step <- -sum(grad0 * d) / sum(d * q_d)
x1 <- x0 + step * d
q_x1 <- drop(Q %*% x1)
grad1 <- q_x1 - b
f0 <- 0.5 * sum(x0 * q_x0) - sum(b * x0)
f1 <- 0.5 * sum(x1 * q_x1) - sum(b * x1)
s <- x1 - x0
y <- grad1 - grad0
```

The step length is 0.25, which moves the iterate to $\underset{\sim}{x}^{(1)} = (0.25, 0.5)$ and lowers $f$ from 0 to -0.625. The difference of the gradients, $\underset{\sim}{y}^{(0)} = (1.5, 1.75)$, is $\mathbf{Q}\underset{\sim}{s}^{(0)}$, which is the secant condition for a quadratic function.

ℹ Remark (Why a quasi-Newton method is used)

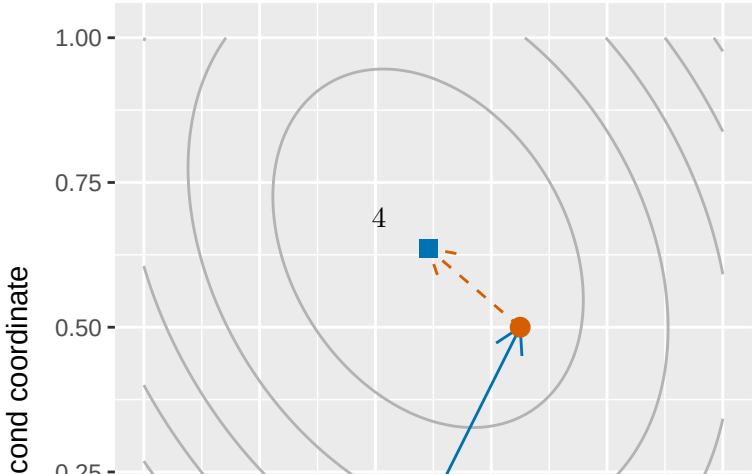
*Remark* 0.1 (Why a quasi-Newton method is used). Because $\mathbf{H}^{(t)}$ is positive definite, $-\mathbf{H}^{(t)}\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)})$ is a descent direction, so, when the gradient is not $\underset{\sim}{0}$, a line search finds a step that decreases $f$. Newton's method needs the Hessian and the solution of a system of equations at every iteration, which costs about $p^3$ operations for a dense $p \times p$ Hessian, and requires the formulas for its entries. A quasi-Newton method needs only the gradient and an update of $\mathbf{H}^{(t)}$ costing about $p^2$ operations. Near a minimizer the method speeds up superlinearly: faster than gradient descent, slower than Newton's method [2, Ch. 6]. Replacing $\mathbf{H}^{(t)}$ by $\mathbf{I}_p$ in the step formula gives gradient descent.

**Definition 0.3** (BFGS update (Broyden-Fletcher-Goldfarb-Shanno))**.** Let $\mathbf{H}$, an approximation of the inverse Hessian, be a $p \times p$ positive definite matrix, and let $\underset{\sim}{s}, \underset{\sim}{y} \in \mathbb{R}^p$ satisfy $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$. With $\rho \overset{\text{def}}{=} 1/\underset{\sim}{y} \cdot \underset{\sim}{s}$, the **BFGS update** [2, Ch. 6] of $\mathbf{H}$ is

$$\mathbf{H}^{+} \overset{\text{def}}{=} (\mathbf{I}_p - \rho \underset{\sim}{s}\underset{\sim}{y}^{\top})\mathbf{H}(\mathbf{I}_p - \rho \underset{\sim}{y}\underset{\sim}{s}^{\top}) + \rho \underset{\sim}{s}\underset{\sim}{s}^{\top}.$$

**R**

```
qn_x_qn <- seq(-0.4, 0.6, length.out = 101)
qn_y_qn <- seq(-0.2, 1, length.out = 101)
qn_grid_qn <- tidyr::expand_grid(x = qn_x_qn, y = qn_y_qn)
qn_gx_qn <- qn_grid_qn$x
qn_gy_qn <- qn_grid_qn$y
qn_quad_qn <- 0.5 * (4 * qn_gx_qn^2 + 2 * qn_gx_qn * qn_gy_qn + 3 * qn_gy_qn^2)
qn_lin_qn <- qn_gx_qn + 2 * qn_gy_qn
qn_grid_qn$z <- qn_quad_qn - qn_lin_qn
qn_star_qn <- c(1, 7) / 11
qn_start_qn <- tibble::tibble(x = 0, y = 0)
qn_next_qn <- tibble::tibble(x = 0.25, y = 0.5)
qn_min_qn <- tibble::tibble(x = qn_star_qn[1], y = qn_star_qn[2])
qn_step_qn <- tibble::tibble(x = 0, y = 0, xend = 0.25, yend = 0.5)
qn_rest_qn <- tibble::tibble(
  x = 0.25, y = 0.5, xend = qn_star_qn[1], yend = qn_star_qn[2]
)
qn_len_qn <- grid::unit(0.15, "inches")
qn_arrow_qn <- grid::arrow(length = qn_len_qn)
qn_aes_qn <- ggplot2::aes(x, y)
qn_seg_qn <- ggplot2::aes(xend = xend, yend = yend)
ggplot2::ggplot(qn_grid_qn, qn_aes_qn) +
  ggplot2::geom_contour(ggplot2::aes(z = z), colour = "grey70", bins = 12) +
  ggplot2::geom_segment(
    data = qn_step_qn, mapping = qn_seg_qn, colour = "#0072B2",
    arrow = qn_arrow_qn
  ) +
  ggplot2::geom_segment(
    data = qn_rest_qn, mapping = qn_seg_qn, colour = "#D55E00",
    linetype = "dashed", arrow = qn_arrow_qn
  ) +
  ggplot2::geom_point(data = qn_start_qn, colour = "#0072B2", size = 3) +
  ggplot2::geom_point(data = qn_next_qn, colour = "#D55E00", size = 3) +
  ggplot2::geom_point(
    data = qn_min_qn, colour = "#0072B2", size = 3, shape = 15
  ) +
  ggplot2::coord_equal() +
  ggplot2::labs(x = "first coordinate", y = "second coordinate")
```



### R

*(This 3D surface is interactive; see the HTML or slide version of this page to view it.)*

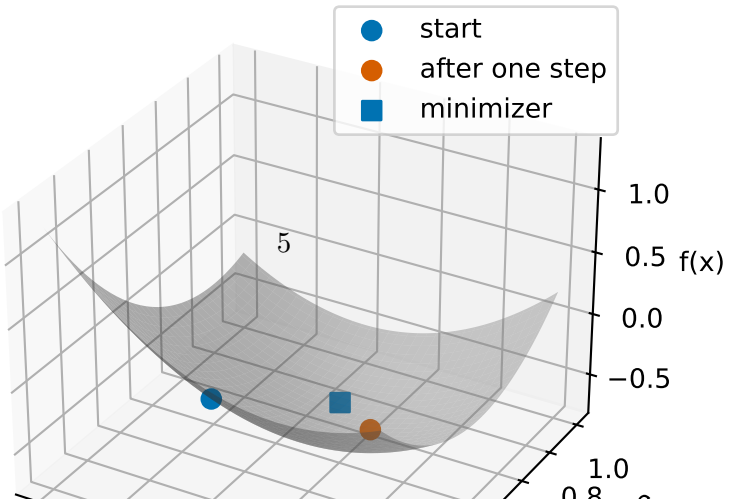
### Python

```
import matplotlib.pyplot as plt
import numpy as np

qn_x_3d = np.linspace(-0.4, 0.6, 101)
qn_y_3d = np.linspace(-0.2, 1, 101)
qn_xx_3d, qn_yy_3d = np.meshgrid(qn_x_3d, qn_y_3d)


def qn_f_3d(x, y):
    return 0.5 * (4 * x**2 + 2 * x * y + 3 * y**2) - x - 2 * y


qn_pts_x_3d = np.array([0, 0.25, 1 / 11])
qn_pts_y_3d = np.array([0, 0.5, 7 / 11])
qn_pts_z_3d = qn_f_3d(qn_pts_x_3d, qn_pts_y_3d)
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(qn_xx_3d, qn_yy_3d, qn_f_3d(qn_xx_3d, qn_yy_3d),
                color="grey", alpha=0.4)
ax.scatter(qn_pts_x_3d[0], qn_pts_y_3d[0], qn_pts_z_3d[0], marker="o",
           color="#0072B2", s=50, label="start")
ax.scatter(qn_pts_x_3d[1], qn_pts_y_3d[1], qn_pts_z_3d[1], marker="o",
           color="#D55E00", s=50, label="after one step")
ax.scatter(qn_pts_x_3d[2], qn_pts_y_3d[2], qn_pts_z_3d[2], marker="s",
           color="#0072B2", s=50, label="minimizer")
ax.set_xlabel("first coordinate")
ax.set_ylabel("second coordinate")
ax.set_zlabel("f(x)")
ax.legend(loc="upper right")
plt.show();
```



**Example 0.3** (A BFGS update from the quasi-Newton step)**.** Continue Example 0.2 with $\mathbf{H} = \mathbf{I}_2$, $\underset{\sim}{s} = \underset{\sim}{s}^{(0)}$, and $\underset{\sim}{y} = \underset{\sim}{y}^{(0)}$.

```
rho <- 1 / sum(y * s)
left <- diag(2) - rho * s %*% t(y)
H1 <- left %*% H0 %*% t(left) + rho * s %*% t(s)
H1
#>         [,1]   [,2]
#> [1,]  0.6625 -0.425
#> [2,] -0.4250  0.650
drop(H1 %*% y)
#> [1] 0.25 0.50
```

Here $\underset{\sim}{y} \cdot \underset{\sim}{s} = 1.25 > 0$, so $\rho = 0.8$. The updated matrix satisfies $\mathbf{H}^{+}\underset{\sim}{y} = (0.25, 0.5)$, which is $\underset{\sim}{s} = (0.25, 0.5)$, and its smallest eigenvalue is 0.231204, so $\mathbf{H}^{+}$ is positive definite.

Remark (The curvature condition $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$)

*Remark* 0.2 (The curvature condition $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$). If the strict inequality of Theorem 4 in Convexity in Several Variables holds for distinct points, then $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$ for $\underset{\sim}{s} = \underset{\sim}{x}^{+} - \underset{\sim}{x} \neq \underset{\sim}{0}$ and $\underset{\sim}{y} = \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{+}) - \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})$. Add the versions at $(\underset{\sim}{x}^{+}, \underset{\sim}{x})$ and at $(\underset{\sim}{x}, \underset{\sim}{x}^{+})$:

$$f(\underset{\sim}{x}^{+}) + f(\underset{\sim}{x}) > f(\underset{\sim}{x}) + f(\underset{\sim}{x}^{+}) + \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) \cdot \underset{\sim}{s} - \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{+}) \cdot \underset{\sim}{s},$$

so $0 > -\underset{\sim}{y} \cdot \underset{\sim}{s}$. For a general $f$, a line search that enforces a curvature condition on the step length (the Wolfe conditions [2, Ch. 3]) guarantees $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$. Such a step can be found along a descent direction when $f$ cannot keep decreasing forever.

**Definition 0.4** (Limited-memory BFGS (L-BFGS))**. Limited-memory BFGS** (**L-BFGS**) [2, Ch. 7] is the quasi-Newton method that never forms $\mathbf{H}^{(t)}$. It stores only the $m$ most recent pairs $(\underset{\sim}{s}^{(j)}, \underset{\sim}{y}^{(j)})$, with $m$ small, such as 5 or 20, and obtains $\mathbf{H}^{(t)} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)})$ implicitly, through a few vector operations, as if the BFGS update for those $m$ pairs were applied to a simple starting matrix, such as a multiple of $\mathbf{I}_p$.

**Example 0.4** (L-BFGS applies the update without forming the matrix)**.**
Continue Example 0.3 with $m = 1$ stored pair $(\underset{\sim}{s}^{(0)}, \underset{\sim}{y}^{(0)})$ and starting matrix $\mathbf{I}_2$. Below, $\mathbf{H}^{+} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(1)})$ is formed from vectors alone, starting from `H0`, then compared with the matrix product.

```
g1 <- grad1
alpha <- rho * sum(s * g1)
q <- g1 - alpha * y
r <- drop(H0 %*% q)
beta <- rho * sum(y * r)
r <- r + s * (alpha - beta)
h1_g1 <- H1 %*% g1
from_matrix <- drop(h1_g1)
tibble::tibble(from_vectors = r, from_matrix = from_matrix)
#> # A tibble: 2 x 2
#>   from_vectors from_matrix
#>          <dbl>       <dbl>
#> 1        0.438       0.438
#> 2       -0.375      -0.375
```

The two columns agree to within 5.6e-17, although the first never forms the matrix. BFGS stores a dense $p \times p$ matrix and spends about $p^2$ operations per iteration; L-BFGS stores $2m$ vectors with $p$ entries and spends about $mp$ operations per iteration, so it remains usable when $p$ is in the millions.


[1] H. Zhou, "Quasi-Newton methods." Lecture notes for Biostat 257, Statistical Computing, University of California, Los Angeles, 2025. Accessed: Oct. 26, 2026. [Online]. Available: https://ucla-biostat-257.github.io/2025spring/slides/24-quasinewton/quasinewton.html

[2] J. Nocedal and S. J. Wright, *Numerical optimization*, 2nd ed. in Springer series in operations research and financial engineering. Springer, 2006. doi: 10.1007/978-0-387-40065-5.